JENKINS DEVOPS PROJECT

CI/CD Pipeline Automation on AWS with Jenkins, GitHub and Jenkins Agent

Student Name: Shubham Gaonkar

Project Type: DevOps / CI-CD Automation

Platform: AWS EC2

Report Date: 15 August 2026

Component	Purpose
AWS EC2	Cloud infrastructure
Ubuntu Linux	Operating system
Jenkins LTS	CI/CD automation server
Jenkins Agent	Build execution node
Git & GitHub	Source control
Node.js / npm	Application dependency and build tooling
SSH	Secure Agent connectivity

1. Project Overview

This project demonstrates a practical DevOps CI/CD implementation using AWS EC2, Ubuntu Linux, Jenkins LTS, GitHub, SSH, and a dedicated Jenkins Agent. The objective was to automate source-code retrieval, dependency installation, application build, testing, packaging, and artifact delivery.

The implementation progressed from Jenkins server installation and a basic Freestyle job to a Jenkins Pipeline executing on a separate Agent. GitHub integration and continuous-integration automation were used to create a repeatable build workflow.

2. Project Objectives

- Provision and configure an Ubuntu EC2 server for Jenkins.
- Install and manage Jenkins LTS and required development tools.
- Integrate Jenkins with a GitHub repository.
- Create and validate a Freestyle project.
- Provision a separate Jenkins Agent EC2 instance and connect it using SSH.
- Create a Declarative Jenkins Pipeline.
- Execute the Pipeline on the Jenkins Agent using a label.
- Automate dependency installation, build, tests, packaging, and artifact delivery.
- Configure GitHub webhook-based continuous integration.
- Create and verify Jenkins backups.

3. AWS Infrastructure

The project uses AWS EC2 instances as the Jenkins Controller and Jenkins Agent. Ubuntu Linux was used as the operating system. Network access was configured through AWS Security Groups.

- Jenkins Controller EC2 instance.
- Jenkins Agent EC2 instance.
- SSH access on port 22 for administration and Agent connectivity.
- Jenkins web access on port 8080.
- Outbound network access required for package installation, GitHub access, and build operations.

GitHub	↓
GitHub Webhook	↓
Jenkins Controller EC2	↓
SSH	↓
Jenkins Agent EC2	↓
Dependencies → Build → Test → Package → Artifact	

4. Task 1 – Jenkins Server Setup

An Ubuntu EC2 instance was prepared as the Jenkins Controller. The operating system was updated, Java was installed, Jenkins LTS was installed and the Jenkins service was enabled.

```
sudo apt update
sudo apt upgrade -y
sudo systemctl start jenkins
sudo systemctl enable jenkins
sudo systemctl status jenkins
```

The Jenkins initial administrator password was retrieved from the Jenkins secrets directory. Jenkins was then accessed through the EC2 public address on port 8080, the setup wizard was completed, suggested plugins were installed, and an administrator account was created.

```
sudo cat /var/lib/jenkins/secrets/initialAdminPassword
```

5. Task 2 – GitHub Integration and Freestyle Job

Git, Node.js, npm, and Java were verified on the Jenkins environment. A GitHub repository was connected to Jenkins and a Freestyle project was configured to check out source code.

- GitHub repository configured as the source repository.
- Successful source-code checkout.
- Freestyle Build #1 executed.
- Console Output verified.
- Build completed with Finished: SUCCESS.
- Freestyle-job evidence captured for documentation.

6. Task 3 – Jenkins Agent and Backup

A second Ubuntu EC2 instance was launched as a dedicated Jenkins Agent. Java was installed and SSH authentication was configured between the Jenkins Controller and Agent.

- Permanent Jenkins Node created.
- Agent label configured as Agent.
- SSH connection established successfully.
- Build execution verified on the Agent.
- Jenkins backup directory created.
- Jenkins backup performed and verified.

```
java --version
ssh ubuntu@
```

7. Task 4 – Jenkins Pipeline Automation

A Declarative Jenkins Pipeline was created to automate the application delivery workflow. The Pipeline was configured to execute on the Jenkins Agent using its label.

```
pipeline {
agent {
label 'Agent'
}

stages {
stage('Clone Source Code') {
steps {
// checkout source code
}
}

stage('Install Dependencies') {
steps {
// install dependencies
}
}

stage('Build Application') {
steps {
// build application
```

```
}
}

stage('Run Tests') {
steps {
// execute tests
}
}

stage('Package Application') {
steps {
// package application
}
}

stage('Deliver Artifact') {
steps {
// archive/deliver artifact
}
}
}
}
```

The Pipeline was debugged through multiple Groovy/Jenkinsfile syntax errors and corrected until the Pipeline executed successfully on the Jenkins Agent.

8. Pipeline Stages

- Clone Source Code – retrieves the application source from GitHub.
- Install Dependencies – installs required application dependencies.
- Build Application – builds the application using the configured tooling.
- Run Tests – executes the project's test stage.
- Package Application – creates the deployable/package artifact.
- Deliver Artifact – archives or delivers the generated build artifact.

9. Task 5 – Continuous Integration

The final CI stage connects GitHub source changes to Jenkins using a webhook. A push to the GitHub repository can trigger the Jenkins Pipeline automatically.

- GitHub webhook configured to communicate with Jenkins.
- Jenkins configured for GitHub push-based triggering.
- A code change is committed and pushed to the repository.
- GitHub sends the webhook event to Jenkins.
- Jenkins starts the Pipeline automatically.
- Console Output is monitored for each stage.
- Successful artifact generation is verified.

```
git add .
git commit -m "Update application"
git push origin main
```

10. Git Workflow

Git was used throughout the project to maintain source-code history and support collaboration between development and CI automation.

```
git init
git add .
git commit -m "Initial project commit"
git branch
git checkout -b project-documentation
git checkout main
git merge project-documentation
git remote -v
git push origin main
```

11. Troubleshooting and Challenges

- Jenkinsfile/Groovy compilation errors were encountered while developing the Declarative Pipeline.
- Pipeline section and stage syntax was corrected to satisfy Jenkins Declarative Pipeline requirements.
- The Jenkins Agent experienced intermittent SSH connection timeouts and required connectivity troubleshooting.
- GitHub authentication and repository synchronization issues were investigated during source-control setup.
- The Jenkins Agent was configured with the correct label so Pipeline builds could execute on the intended node.
- Jenkins backup and Agent connectivity were verified as part of infrastructure administration.

12. Security and Administration

- SSH was used for secure remote administration and Jenkins Agent connectivity.
- AWS Security Groups were used to control inbound access.
- Jenkins service status was verified with systemctl.
- Jenkins was configured to start automatically with the operating system.
- Backups were created for Jenkins data/configuration as part of operational readiness.

13. Final Architecture

The completed workflow is represented below:

```
Developer
|
| git push
v
GitHub Repository
|
| Webhook
v
Jenkins Controller (AWS EC2)
|
| SSH
v
Jenkins Agent (AWS EC2)
|
+--> Clone Source Code
+--> Install Dependencies
+--> Build Application
+--> Run Tests
+--> Package Application
+--> Deliver Artifact
|
v
Successful CI Build / Artifact
```

14. Project Deliverables

- AWS Jenkins Controller EC2.

- AWS Jenkins Agent EC2.
- Jenkins LTS installation and configuration.
- GitHub repository integration.
- Freestyle Jenkins job.
- Jenkinsfile / Declarative Pipeline.
- GitHub webhook CI trigger.
- Successful Pipeline execution on Agent.
- Build artifact.
- Jenkins backup.

15. Learning Outcomes

- AWS EC2 infrastructure administration.
- Ubuntu Linux administration and package management.
- Jenkins LTS installation and service management.
- Jenkins Controller and Agent architecture.
- SSH-based Jenkins Agent connectivity.
- Git and GitHub source-code management.
- Declarative Jenkins Pipeline development.
- CI automation using GitHub webhooks.
- Build, test, package, and artifact workflows.
- Troubleshooting Jenkins and Groovy configuration errors.
- Jenkins backup and operational maintenance.

16. Conclusion

The project demonstrates a complete foundational CI/CD workflow using AWS, Ubuntu, GitHub, Jenkins, and a dedicated Jenkins Agent. It progressed from infrastructure preparation and basic Jenkins jobs to Pipeline-based automation and continuous integration. The implementation provides practical experience with the tools, troubleshooting techniques, and operational practices expected of a Junior DevOps Engineer.